PES UNIVERSITY
100 Feet Ring Road, BSK 3rd Stage, Bengaluru – 560085

UE23CS351A: DBMS Project Aug–Dec 2026

SMARTCIVIC

Civic Incident Management and Similarity Search Platform

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)
B.Tech. CSE – 5th Semester
Aug–Dec 2026

DATABASE MANAGEMENT SYSTEMS (DBMS)
PROJECT REPORT

Submitted by

Sri Ram V | PES1UG24AM283
Shreyan Ganesh | PES1UG25AM813

Class of Prof. Mir Junaid Rasool

Report prepared: 12 November 2026

Abstract

SmartCivic is a civic incident management platform designed to make it easier for residents to report local problems and follow the progress of their reports. Issues such as potholes, overflowing garbage bins, water leakage and non-functional streetlights can be submitted through a resident portal. Administrators review incoming reports, set priorities, route them to the appropriate department and assign officers. Officers work on their assigned incidents and update the status, while residents can follow the recorded activity.

The application uses PostgreSQL as its main relational database. It stores user accounts, profiles, departments, categories, incidents, assignments, status history, feedback, complaints and notifications. Database constraints help protect data consistency, while transactions keep related changes together. PostgreSQL full-text search supports keyword-based incident search. Qdrant is used as a separate vector database for similarity search over incident descriptions. The current vectorizer creates 384-dimensional vectors using deterministic lexical feature hashing; it is not a pretrained language-model embedding.

The current implementation also supports resident complaints for unresolved incidents. Complaint submissions are recorded with an activity-history entry and notifications for administrators and assigned officers. The administrator interface provides a combined feedback and escalation view, and notification panels are available in the administrator and officer portals.

Table of Contents

Sl. No.	Topic	Page No.
1	Introduction and Problem Statement	4
2	User Requirements Specification – Tech Stack	5
3	E-R Model	6
4	Relational Model	8
5	SQL DDL Statements	9
6	SQL DML Statements	12
7	NoSQL / Vector Database Commands	15
8	Results	18
9	Next.js Project Tree Structure	19
10	Sample Screenshots of Application Code and Demo	20
11	Evaluation Rubrics – Completion Status	25

1. Introduction

1.1 Project Overview

SmartCivic brings residents, administrators and civic department officers into one workflow. Instead of treating a report as a standalone form submission, the system follows it through review, assignment, progress updates and closure. The application maintains the relationship between an incident category and the department responsible for it, so that reports can be routed to the right team.

1.2 Problem Statement

Residents often have limited visibility after reporting a civic issue. A report may be submitted, but the resident may not know which department is handling it, whether an officer has been assigned or why the issue is taking time to resolve. Administrators also need a consistent way to organize incoming reports, prioritize urgent cases and monitor departmental workload.

The problem addressed by SmartCivic is to design and implement a database-backed application that records civic incidents, routes them to the appropriate department, supports officer assignment and status tracking, preserves an activity history, and gives residents a way to provide feedback or raise a complaint when an unresolved issue is not progressing.

1.3 Main Objectives

- Allow residents to create accounts, submit civic incidents and track their progress.
- Maintain departments and incident categories, with each category mapped to a responsible department.
- Allow administrators to review incidents, set priority and assign officers.
- Restrict officers to the work and department data they are authorized to access.
- Record every incident status change in a history table.
- Support feedback for resolved incidents and complaints for unresolved incidents.
- Notify administrators and assigned officers about new feedback and complaints.
- Provide PostgreSQL full-text search and Qdrant similarity search for finding related incidents.

1.4 Scope and User Roles

Role	Main responsibilities
Resident / Civic user	Create an account, report an incident, view own reports and activity, search for similar incidents, submit feedback after resolution and raise a complaint for an unresolved incident.
Administrator	Review all incoming reports, set priority, manage departments and officers, assign incidents, review feedback and complaints, and receive notifications.
Officer	View incidents permitted by department and assignment rules, update authorized incident statuses and receive relevant notifications.

2. User Requirements Specification – Tech Stack

2.1 Functional Requirements

- The system must authenticate users and distinguish resident, administrator and officer roles.
- Residents must be able to submit incident title, description, category, priotity and location.
- Administrators must be able to manage departments, categories, managers and officers.
- Incidents must support priority and status values, and assignment to officers.
- Status transitions must be recorded with the actor, old status, new status, remarks and timestamp.
- Residents may submit feedback for eligible resolved incidents, with a rating from 1 to 5.
- Residents may submit a complaint for an incident that is not resolved; the complaint must belong to the signed-in resident.
- Administrators and assigned officers must be able to view relevant feedback/complaint notifications and mark their own notifications as read.
- Similar-incident search must only return incidents the current user is authorized to access.

2.2 Non-Functional Requirements

- Data integrity: primary keys, foreign keys, CHECK constraints and UNIQUE constraints protect important records.
- Security: passwords are hashed, sessions are signed and role checks are performed on the server.
- Reliability: related database writes use transactions where partial completion would leave inconsistent records.
- Maintainability: database changes are organized as numbered SQL migrations.
- Search: PostgreSQL supports lexical full-text search and Qdrant supports vector similarity ranking.

2.3 Technology Stack and Justification

Layer	Technology	Why it is used
Frontend and routing	Next.js App Router, React, TypeScript	Provides role-specific pages and API Route Handlers in one application.
Relational database	PostgreSQL	Stores the authoritative application records and enforces relationships, constraints and transactions.
Database access	node-postgres (pg)	Provides connection pooling and parameterized SQL queries.
Vector database	Qdrant	Stores incident vectors and performs nearest-neighbour search using cosine distance.
Vector generation	smartcivic-feature-hash-v1	Produces deterministic 384-dimensional vectors from tokens and adjacent-token features.
Authentication	bcryptjs, jose, HTTP-only cookies	Hashes passwords and manages signed sessions.
Local service management	Docker Compose	Runs Qdrant locally with persistent storage.

2.4 Why PostgreSQL and Qdrant?

PostgreSQL is suitable for the core application because incidents have clear relationships with residents, categories, departments, assignments and history records. These relationships need foreign keys, uniqueness rules, transactions and authorization checks. PostgreSQL also provides full-text search through TSVECTOR, a trigger to maintain the search vector, and a GIN index to support text matching.

Qdrant is used for a different purpose: finding incidents with similar text. It stores a 384-dimensional vector and uses cosine distance to rank nearby vectors. PostgreSQL remains the source of truth. Before Qdrant is queried, the application obtains the incident IDs the current user is allowed to access from PostgreSQL. This keeps access control in the relational database.

The current vectorizer is deterministic feature hashing, not a pretrained semantic embedding model. It lowercases text, extracts tokens and adjacent-token pairs, hashes them into 384 buckets and normalizes the vector. This is a practical lightweight approach for the current project, although it may not understand paraphrases as well as a pretrained language model.

3. E-R Model

3.1 List of Entities

Entity	Purpose	Weak entity?
Auth_Accounts	Stores login identity, email and password hash.	No
Users	Stores application profile and role; links to an authentication account.	No
Departments	Represents civic departments such as Public Works and Sanitation.	No
Categories	Represents incident types and maps each type to a department.	No
Incidents	Stores the civic problem reported by a resident.	No
Assignments	Associates incidents with officers and the administrator who assigned them.	No
Status_History	Stores each incident status transition and its actor.	No
Feedback	Stores resident ratings/comments for resolved incidents.	No
Complaints	Stores resident escalation details for unresolved incidents.	No
Notifications	Stores notification records for administrators and assigned officers.	No
Incident_Vector_Sync	Tracks whether an incident vector has been indexed in Qdrant.	No

The current schema does not define a weak entity in the strict ER-model sense. The workflow tables depend on parent records through foreign keys, but they each have their own primary key (except incident_vector_sync, which uses incident_id as both primary key and foreign key).

3.2 Non-Simple Attributes

Attribute	Entity	Explanation
location	Incidents	A single stored text value containing the incident location; it may be treated as composite conceptually if split into address components.
created_at / updated_at	Incidents	Timestamp attributes that record creation and latest modification time.
old_status / new_status	Status_History	Capture the transition from one status value to another.
reason	Complaints	Stores the resident's explanation for escalating the incident.
rating	Feedback	Numeric rating constrained to the range 1–5.
search_vector	Incidents	PostgreSQL TSVECTOR derived from title, description and location.

3.3 ER Diagram Snapshot

The conceptual ER diagram below is retained from the supplied project design. It shows the original core entities and their relationships.

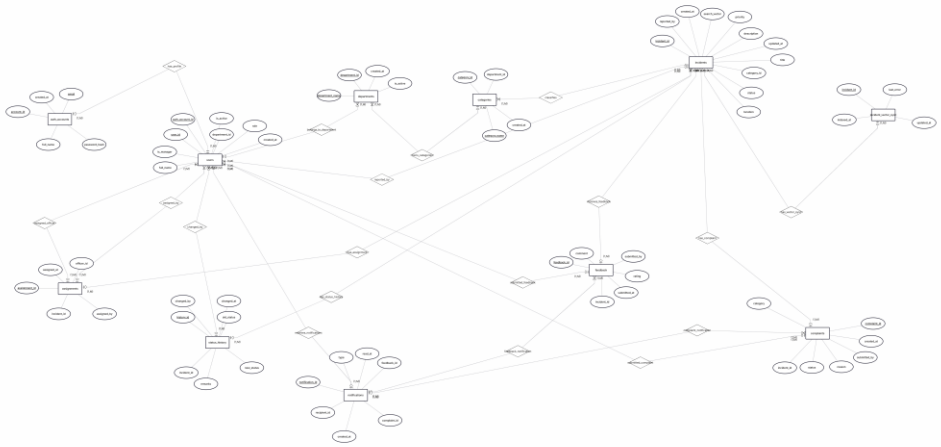


Figure 1. Conceptual ER diagram for SmartCivic.

3.4 Relational ER Diagram

The relational diagram shows the complaints and notifications relations alongside the core incident workflow tables.

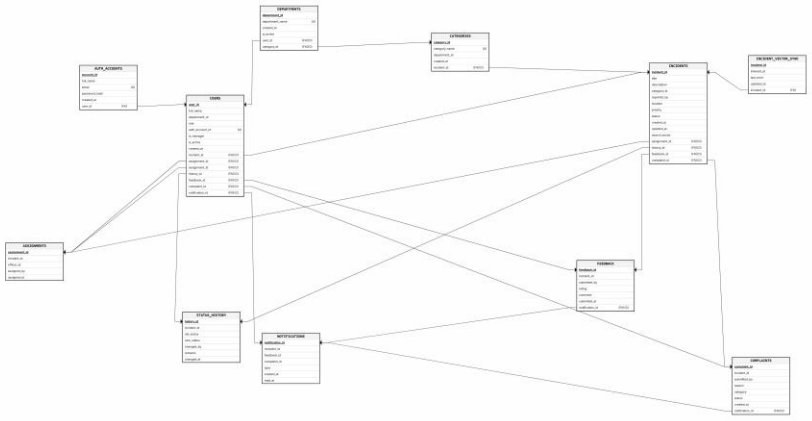


Figure 2. Relational schema with complaints and notifications.

4. Relational Model

4.1 Relations that are Additional to the Main ER Entities

The implementation includes operational and support relations that may be shown as associative or workflow relations in the ER model. These include assignments, status_history, feedback, complaints, notifications and incident_vector_sync. The current schema uses a primary key for each of these relations, except incident_vector_sync, whose incident_id is both its primary key and a foreign key to incidents.

4.2 Relational Schema and Data Dictionary

Relation	Important attributes and constraints
auth_accounts	account_id PK; full_name; email UNIQUE and lowercase CHECK; password_hash; created_at.
users	user_id PK; auth_account_id FK and UNIQUE; department_id FK; role CHECK; is_manager; is_active.
departments	department_id PK; department_name UNIQUE; is_active; created_at.
categories	category_id PK; category_name UNIQUE; department_id FK.
incidents	incident_id PK; title; description; location; category_id FK; reported_by FK; priority CHECK; status CHECK; timestamps; search_vector TSVECTOR.
assignments	assignment_id PK; incident_id FK; officer_id FK; assigned_by FK; assigned_at; UNIQUE(incident_id, officer_id).
status_history	history_id PK; incident_id FK; old_status; new_status; changed_by FK; remarks; changed_at.
incident_vector_sync	incident_id PK/FK; indexed_at; last_error; updated_at.
feedback	feedback_id PK; incident_id FK; submitted_by FK; rating CHECK 1–5; UNIQUE(incident_id, submitted_by).
complaints	complaint_id PK; incident_id FK; submitted_by FK; reason; category; status; created_at.
notifications	notification_id PK; recipient_id FK; feedback_id nullable FK; complaint_id nullable FK; type; created_at; read_at.
department_incident_summary	VIEW: department name, total incidents, open incidents and resolved

Relation	Important attributes and constraints
	incidents.

4.3 Relationships and Integrity Constraints

- One authentication account can link to at most one application user profile because users.auth_account_id is unique.
- A department can have multiple officers and categories. A category belongs to one department.
- Each incident references one category and one reporting user.
- An incident can have multiple assignments, status-history entries, feedback records and complaints.
- incident_vector_sync tracks one synchronization state per incident.
- A notification belongs to one recipient and may refer to feedback or a complaint.
- A partial unique index permits at most one officer marked as manager in a department.
- Foreign-key delete rules use CASCADE, RESTRICT or SET NULL according to whether dependent records should be removed, protected or retained with an optional link.

5. SQL DDL Statements

The database is built through ordered migration files. DDL defines or changes the structure of the database. In SmartCivic, CREATE TABLE defines relations, ALTER TABLE adds fields, PRIMARY KEY identifies rows, FOREIGN KEY preserves relationships, CHECK validates allowed values, UNIQUE prevents duplicates, and CREATE INDEX improves lookup performance.

5.1 Current Migration Sequence

Migration	Purpose
001_create_auth_accounts.sql	Creates authentication accounts with unique lowercase email and password hash.
002_create_incident_relations.sql	Creates departments, users, categories, incidents, status_history and assignments; inserts initial department/category/officer seed data.
003_department_officer_managers.sql	Adds is_manager, backfills a manager where possible and creates a partial unique index for one officer-manager per department.
004_project_features.sql	Adds active flags, incident search_vector, trigger/function, GIN and B-tree indexes and department_incident_summary view.
005_vector_search_outbox.sql	Creates incident_vector_sync and seeds existing incident IDs for synchronization tracking.
006_feedback.sql	Creates feedback with rating CHECK 1–5 and one-feedback-per-resident-per-incident uniqueness.
007_notifications.sql	Creates notifications initially associated with feedback.
008_backfill_feedback_notifications.sql	Backfills feedback notifications for active administrators and assigned officers.
009_complaints.sql	Creates complaints and extends notifications to support complaint notifications.

5.2 Core DDL — Authentication Accounts

```
CREATE TABLE IF NOT EXISTS auth_accounts (
  account_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
```

```
  email VARCHAR(254) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT auth_accounts_email_lowercase
    CHECK (email = LOWER(email))
);
```

This table keeps login credentials separate from the application profile. The email is unique and must be lowercase. Passwords are stored as password hashes, not plaintext.

5.3 Core DDL — Incident Relations

```
CREATE TABLE departments (
  department_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  department_name VARCHAR(120) NOT NULL UNIQUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE users (
  user_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  department_id BIGINT REFERENCES departments(department_id)
    ON DELETE SET NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'resident'
    CHECK (role IN ('resident', 'officer', 'admin')),
  auth_account_id BIGINT UNIQUE
    REFERENCES auth_accounts(account_id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE categories (
  category_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  category_name VARCHAR(120) NOT NULL UNIQUE,
  department_id BIGINT NOT NULL
    REFERENCES departments(department_id) ON DELETE RESTRICT
);

CREATE TABLE incidents (
  incident_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  description VARCHAR(5000) NOT NULL,
  category_id BIGINT NOT NULL
    REFERENCES categories(category_id) ON DELETE RESTRICT,
  reported_by BIGINT NOT NULL
    REFERENCES users(user_id) ON DELETE RESTRICT,
  location VARCHAR(300) NOT NULL,
  priority VARCHAR(10) NOT NULL DEFAULT 'medium'
    CHECK (priority IN ('low','medium','high','critical')),
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
```

```
    CHECK (status IN ('pending','assigned','in_progress',
                      'resolved','rejected')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

5.4 Manager, Search, Indexes and View

```
ALTER TABLE users
  ADD COLUMN IF NOT EXISTS is_manager BOOLEAN NOT NULL DEFAULT false;

CREATE UNIQUE INDEX users_one_officer_manager_per_department
  ON users (department_id)
  WHERE role = 'officer' AND is_manager = true;

ALTER TABLE incidents ADD COLUMN IF NOT EXISTS search_vector TSVECTOR;

CREATE OR REPLACE FUNCTION maintain_incident_search_vector()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  NEW.search_vector := to_tsvector(
    'english',
    coalesce(NEW.title,'') || ' ' ||
    coalesce(NEW.description,'') || ' ' ||
    coalesce(NEW.location,'')
  );
  NEW.updated_at := CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$;

CREATE TRIGGER incidents_search_vector_update
BEFORE INSERT OR UPDATE OF title, description, location ON incidents
FOR EACH ROW EXECUTE FUNCTION maintain_incident_search_vector();

CREATE INDEX incidents_search_vector_gin
  ON incidents USING GIN (search_vector);

CREATE OR REPLACE VIEW department_incident_summary AS
SELECT d.department_id, d.department_name,
       COUNT(i.incident_id)::BIGINT AS total_incidents,
       COUNT(i.incident_id) FILTER
         (WHERE i.status NOT IN ('resolved','rejected'))::BIGINT
         AS open_incidents,
       COUNT(i.incident_id) FILTER
         (WHERE i.status = 'resolved')::BIGINT AS resolved_incidents
FROM departments d
LEFT JOIN categories c ON c.department_id = d.department_id
```

```
LEFT JOIN incidents i ON i.category_id = c.category_id
GROUP BY d.department_id, d.department_name;
```

The trigger function updates search_vector whenever the incident title, description or location changes. The GIN index supports full-text matching. The view provides department-level workload counts.

5.5 Feedback, Complaints and Notifications DDL

```
CREATE TABLE feedback (
  feedback_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  incident_id BIGINT NOT NULL
    REFERENCES incidents(incident_id) ON DELETE CASCADE,
  submitted_by BIGINT NOT NULL
    REFERENCES users(user_id) ON DELETE RESTRICT,
  rating SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
  submitted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (incident_id, submitted_by)
);

CREATE TABLE complaints (
  complaint_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  incident_id BIGINT NOT NULL
    REFERENCES incidents(incident_id) ON DELETE CASCADE,
  submitted_by BIGINT NOT NULL
    REFERENCES users(user_id) ON DELETE RESTRICT,
  reason TEXT NOT NULL,
  category VARCHAR(50) DEFAULT 'general',
  status VARCHAR(30) NOT NULL DEFAULT 'open',
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE notifications (
  notification_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  recipient_id BIGINT NOT NULL
    REFERENCES users(user_id) ON DELETE CASCADE,
  feedback_id BIGINT
    REFERENCES feedback(feedback_id) ON DELETE CASCADE,
  complaint_id BIGINT
    REFERENCES complaints(complaint_id) ON DELETE CASCADE,
  type VARCHAR(30) NOT NULL DEFAULT 'feedback',
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  read_at TIMESTAMPTZ
);
```

In the migration sequence, notifications are first created for feedback in migration 007. Migration 009 then makes feedback_id nullable and adds complaint_id and type. The current schema does not yet

enforce a CHECK constraint that exactly one of feedback_id or complaint_id must be populated; this can be considered as a future integrity improvement.

6. SQL DML Statements

DML statements insert, read, update and delete table rows. SmartCivic uses parameterized PostgreSQL queries through node-postgres. Placeholders such as $1 and $2 are bound separately from the SQL text. This keeps user-provided values out of the SQL command string and helps prevent SQL injection.

6.1 INSERT — Seed Departments and Categories

```
INSERT INTO departments (department_id, department_name)
OVERRIDING SYSTEM VALUE
VALUES (1,'Public Works'), (2,'Sanitation'),
       (3,'Water Services'), (4,'Electrical Services')
ON CONFLICT (department_id) DO UPDATE
SET department_name = EXCLUDED.department_name;

INSERT INTO categories (category_id, category_name, department_id)
OVERRIDING SYSTEM VALUE
VALUES (1,'Pothole',1), (2,'Garbage Overflow',2),
       (3,'Water Leakage',3), (4,'Street Light Failure',4)
ON CONFLICT (category_id) DO UPDATE
SET category_name = EXCLUDED.category_name,
    department_id = EXCLUDED.department_id;
```

These statements create the initial departments and categories. ON CONFLICT makes the seed operation safe to repeat for the supplied IDs.

6.2 CTE and NOT EXISTS — Manager Backfill

```
WITH departments_without_managers AS (
  SELECT d.department_id, MIN(u.user_id) AS user_id
  FROM departments d
  JOIN users u ON u.department_id = d.department_id
              AND u.role = 'officer'
  WHERE NOT EXISTS (
    SELECT 1 FROM users manager
    WHERE manager.department_id = d.department_id
      AND manager.role = 'officer'
      AND manager.is_manager = true
  )
  GROUP BY d.department_id
)
UPDATE users u
SET is_manager = true
FROM departments_without_managers candidate
WHERE u.user_id = candidate.user_id;
```

The CTE identifies departments that have officers but no manager. It selects one officer profile and marks that profile as the manager. NOT EXISTS checks that no manager already exists.

6.3 INSERT — Create a New Incident

```
INSERT INTO incidents
  (title, description, category_id, reported_by, location, priority)
VALUES ($1, $2, $3, $4, $5, $6)
RETURNING incident_id, title, status, created_at;
```

The API validates the signed-in resident and selected category before inserting the report. The initial status-history and vector-sync records are handled as part of the workflow.

6.4 SELECT with JOIN — Administrator Incident List

```
SELECT i.incident_id, i.title, i.priority, i.status,
       c.category_name, d.department_name,
       reporter.full_name AS reported_by
FROM incidents i
JOIN categories c ON c.category_id = i.category_id
JOIN departments d ON d.department_id = c.department_id
JOIN users reporter ON reporter.user_id = i.reported_by
WHERE i.status <> 'resolved'
ORDER BY CASE i.priority
  WHEN 'critical' THEN 1 WHEN 'high' THEN 2
  WHEN 'medium' THEN 3 WHEN 'low' THEN 4 ELSE 5 END,
  i.created_at DESC;
```

The JOIN operations replace foreign-key IDs with readable category, department and resident names. CASE applies the priority order before the creation-time sort.

6.5 UPDATE, FOR UPDATE and Transaction

```
BEGIN;
SELECT status FROM incidents
WHERE incident_id = $1
FOR UPDATE;

UPDATE incidents
SET status = $2, updated_at = CURRENT_TIMESTAMP
WHERE incident_id = $1;

INSERT INTO status_history
  (incident_id, old_status, new_status, changed_by, remarks)
VALUES ($1, $3, $2, $4, $5);
COMMIT;
```

The application checks the actor's role, department and assignment before applying the status change. FOR UPDATE locks the incident row so another transaction cannot modify it at the same time. The status update and audit-history insert are committed together.

6.6 GROUP BY, HAVING and NOT EXISTS

```
SELECT d.department_name, COUNT(i.incident_id) AS total_incidents
FROM departments d
LEFT JOIN categories c ON c.department_id = d.department_id
LEFT JOIN incidents i ON i.category_id = c.category_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(i.incident_id) > 0
ORDER BY total_incidents DESC;

SELECT i.incident_id, i.title
FROM incidents i
WHERE i.reported_by = $1 AND i.status = 'resolved'
AND NOT EXISTS (
  SELECT 1 FROM feedback f
  WHERE f.incident_id = i.incident_id
    AND f.submitted_by = $1
);
```

GROUP BY forms one row per department and HAVING filters grouped results. NOT EXISTS finds resolved incidents for which the resident has not yet submitted feedback.

6.7 Complaint Submission — Transactional Workflow

```
BEGIN;

INSERT INTO complaints
  (incident_id, submitted_by, reason, category)
VALUES ($1, $2, $3, $4)
RETURNING complaint_id;

INSERT INTO status_history
  (incident_id, old_status, new_status, changed_by, remarks)
VALUES ($1, $5, $5, $2, $3);

-- Notification rows are created for active administrators
-- and officers assigned to the incident.

COMMIT;
```

The complaint API verifies that the incident belongs to the authenticated resident and is not resolved. The complaint, audit-history entry and notifications are written within one transaction. The incident's own status is not changed; the history row records the complaint event and reason.

6.8 Full-Text Search Query

```
SELECT incident_id, title, status,
       ts_rank(search_vector,
               plainto_tsquery('english', $1)) AS rank
FROM incidents
WHERE search_vector @@ plainto_tsquery('english', $1)
```

```
ORDER BY rank DESC, created_at DESC
LIMIT 20;
```

This query uses PostgreSQL lexical full-text search. It is separate from Qdrant vector similarity search.

7. NoSQL / Vector Database Commands

Qdrant is used to find incidents with similar text. PostgreSQL stores the full incident record and controls which incident IDs a user may access. Qdrant only ranks vectors; it does not replace SQL authorization.

Setting	Project value
Collection	smartcivic_incidents
Vector dimension	384
Distance	Cosine
Vectorizer	smartcivic-feature-hash-v1
Features	Lowercase Unicode tokens and adjacent-token bigrams, hashed into buckets and L2-normalized.
Payload	incident_id and embedding_model identifier.
Sync state	incident_vector_sync: indexed_at, last_error and updated_at.

7.1 Vector Generation

```
export const INCIDENT_EMBEDDING_DIMENSIONS = 384;
export const INCIDENT_EMBEDDING_MODEL =
  "smartcivic-feature-hash-v1";

export function embedIncidentText(text: string): number[] {
  const tokens = text.toLowerCase()
    .match(/[\p{L}\p{N}]+/gu)
    ?.filter((token) => token.length > 1) ?? [];

  const features = [...tokens];
  for (let i = 0; i < tokens.length - 1; i += 1) {
    features.push(`${tokens[i]}_${tokens[i + 1]}`);
  }

  const vector = Array<number>(384).fill(0);
  for (const feature of features) {
    const hash = hashFeature(feature);
    const bucket = hash % 384;
    vector[bucket] += hash & 1 ? 1 : -1;
  }
  // L2-normalize the vector before returning it.
  return normalize(vector);
}
```

The vectorizer tokenizes the text, adds adjacent-token pairs, maps features to fixed buckets and normalizes the result. This is deterministic lexical feature hashing, not a pretrained sentence-transformer embedding.

7.2 Create Qdrant Collection

```
PUT /collections/smartcivic_incidents
Content-Type: application/json

{
  "vectors": {
    "size": 384,
    "distance": "Cosine"
  }
}
```

7.3 Upsert an Incident Vector

```
PUT /collections/smartcivic_incidents/points?wait=true
Content-Type: application/json

{
  "points": [{
    "id": 42,
    "vector": ["<384 normalized floating-point values>"],
    "payload": {
      "incident_id": "42",
      "embedding_model": "smartcivic-feature-hash-v1"
    }
  }]
}
```

7.4 Search Similar Incidents

```
POST /collections/smartcivic_incidents/points/search
Content-Type: application/json

{
  "vector": ["<384 query values>"],
  "filter": {
    "must": [{ "has_id": [42, 43, 44] }]
  },
  "limit": 20,
  "with_payload": true,
  "with_vectors": false
}
```

7.5 Delete a Vector

```
POST /collections/smartcivic_incidents/points/delete?wait=true
Content-Type: application/json
```

```
{ "points": [42] }
```

The search filter is built from IDs that PostgreSQL has already authorized for the current user. The application then fetches incident details from PostgreSQL. The incident_vector_sync table tracks indexing state and any synchronization error.

8. Results

The current implementation provides separate resident, administrator and officer workflows. The screenshots in Section 10 show the main interface pages and the latest complaint/notification changes. The following table summarizes the database outputs and their purpose.

Database object	Expected contents / result	How it supports the project
departments	Public Works, Sanitation, Water Services, Electrical Services and administrator-created departments.	Groups incident categories and officer responsibility.
categories	Pothole, Garbage Overflow, Water Leakage, Street Light Failure and additional configured categories.	Routes a report to its responsible department.
incidents	Resident reports with title, description, category, location, priority and status.	Main record tracked through the workflow.
assignments	Incident-to-officer assignment rows.	Shows who is responsible for working on an incident.
status_history	Old/new status, actor, remarks and timestamp.	Provides an audit trail for progress and escalation events.
feedback	Rating and optional comment for eligible resolved incidents.	Captures resident satisfaction.
complaints	Reason, category, complaint status and submitter for unresolved incidents.	Allows residents to escalate reports that are not progressing.
notifications	Recipient, feedback/complaint reference, type and read timestamp.	Alerts administrators and assigned officers.

8.1 Result Verification Notes

The source archive and supplied application screenshots show the current feature set. No separate live PostgreSQL table-result screenshots were supplied with this report update, so this section does not invent database row counts or claim that a particular SQL query returned specific live records. The database tables can be displayed during the demonstration using SELECT queries against the configured database.

8.2 Useful Queries for Demonstration

```
-- Show the departments
SELECT * FROM departments ORDER BY department_id;

-- Show incidents with category and department names
SELECT i.incident_id, i.title, c.category_name,
       d.department_name, i.priority, i.status
FROM incidents i
JOIN categories c ON c.category_id = i.category_id
JOIN departments d ON d.department_id = c.department_id
```

```
ORDER BY i.created_at DESC;

-- Show complaints and the incident they refer to
SELECT cp.complaint_id, cp.incident_id, cp.reason,
       cp.category, cp.status, cp.created_at
FROM complaints cp
ORDER BY cp.created_at DESC;

-- Show unread notifications
SELECT notification_id, recipient_id, type,
       feedback_id, complaint_id, created_at
FROM notifications
WHERE read_at IS NULL
ORDER BY created_at DESC;
```

9. Next.js – Project Tree Structure

```
smart-incident-platform/
├── app/
│   ├── api/
│   │   ├── auth/                 # signup, login, logout, me
│   │   ├── incidents/            # CRUD, status, similarity search
│   │   ├── assignments/
│   │   ├── categories/
│   │   ├── departments/
│   │   ├── officers/
│   │   ├── feedback/
│   │   ├── complaints/
│   │   ├── notifications/
│   │   └── activity-history/
│   ├── admin/                    # administrator pages
│   ├── civic/                    # resident dashboard and history
│   ├── officer/                  # officer portal
│   └── login/
├── components/                   # portal and workflow components
├── db/migrations/                # PostgreSQL migrations 001–009
├── lib/
│   ├── auth/                     # session and authentication
│   ├── db.ts                     # PostgreSQL connection pool
│   ├── incident-embeddings.ts
│   ├── incident-vector-sync.ts
│   └── qdrant.ts
├── docker-compose.yml
├── next.config.ts
└── package.json
```

File / module	Responsibility
lib/db.ts	Creates the PostgreSQL connection pool.
lib/auth/session.ts	Signs and verifies session tokens and configures the HTTP-only session cookie.
lib/auth/authenticate.ts	Checks the current role, account activity and department activity.
lib/incident-embeddings.ts	Creates 384-dimensional feature-hash vectors.
lib/incident-vector-sync.ts	Upserts vectors and records synchronization state/errors.
lib/qdrant.ts	Creates the Qdrant collection and performs upsert, search, delete and health checks.
app/api/complaints	Validates and stores resident complaints; provides role-scoped retrieval.
app/api/notifications	Returns recipient-scoped notifications and updates read state.
app/api/feedback	Stores resident feedback and provides administrative feedback data.
app/admin/feedback	Displays feedback and complaint escalation information.
components/notifications-panel.tsx	Displays feedback/complaint alerts and mark-as-read action.

10. Sample Screenshots of Application Code and Demo

This section brings the supplied application screenshots together in one place, as requested by the project template. The original resident, department and activity-history screenshots are retained. Screens that changed in the current implementation are represented by the newer screenshots captured on 12 November 2026.

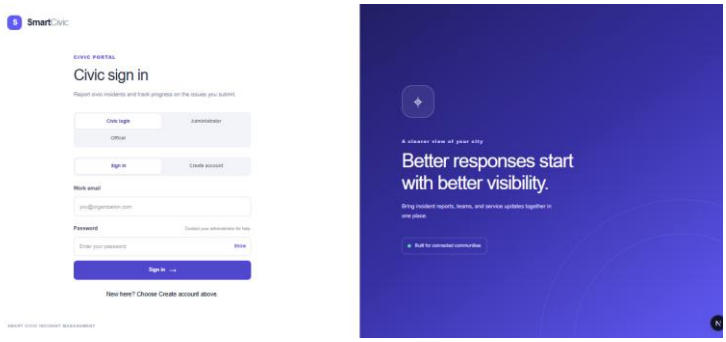


Figure 3. Civic portal sign-in with Civic, Administrator and Officer portal selection and account creation.

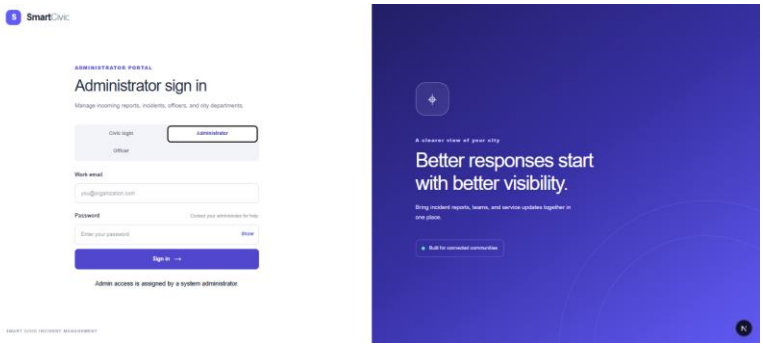


Figure 4. Administrator sign-in page. Administrator access is assigned by a system administrator.

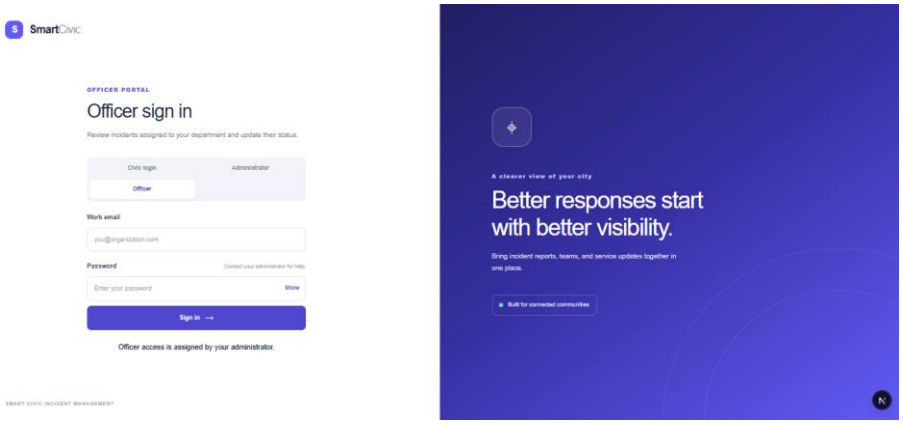


Figure 5. Officer sign-in page. Officer access is assigned by a system administrator.

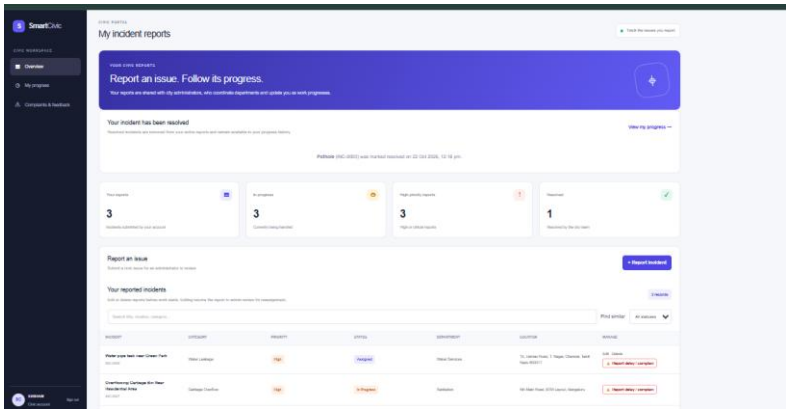


Figure 6. Resident dashboard — incident reporting, summary cards and reported incidents.

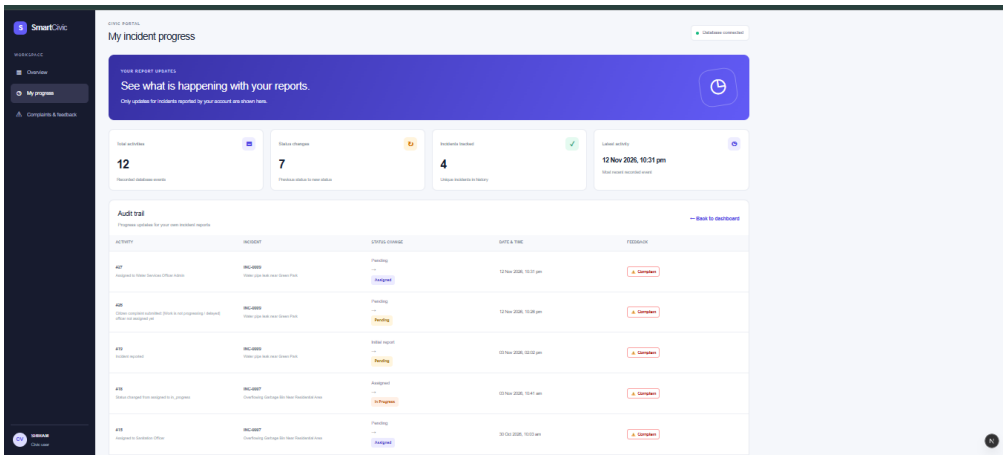
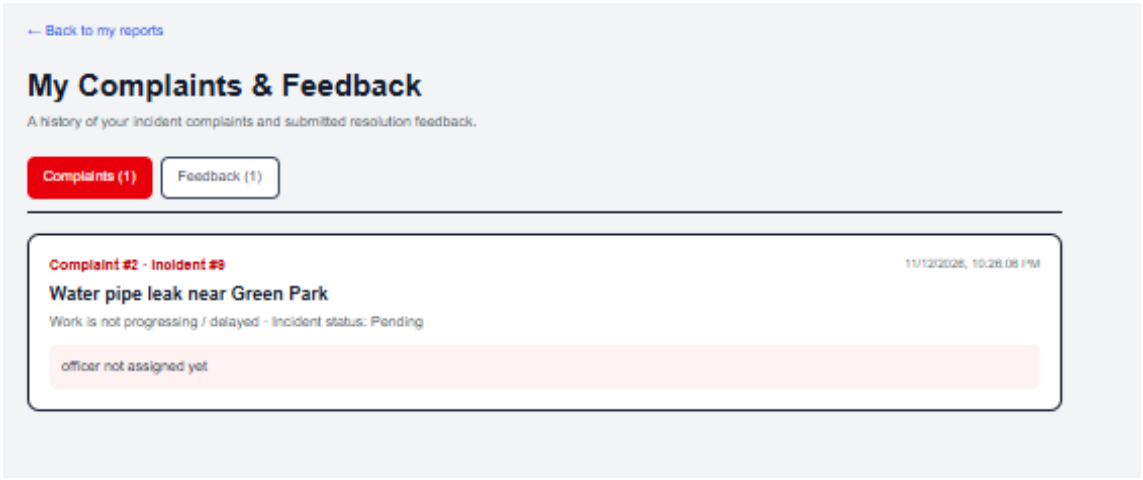
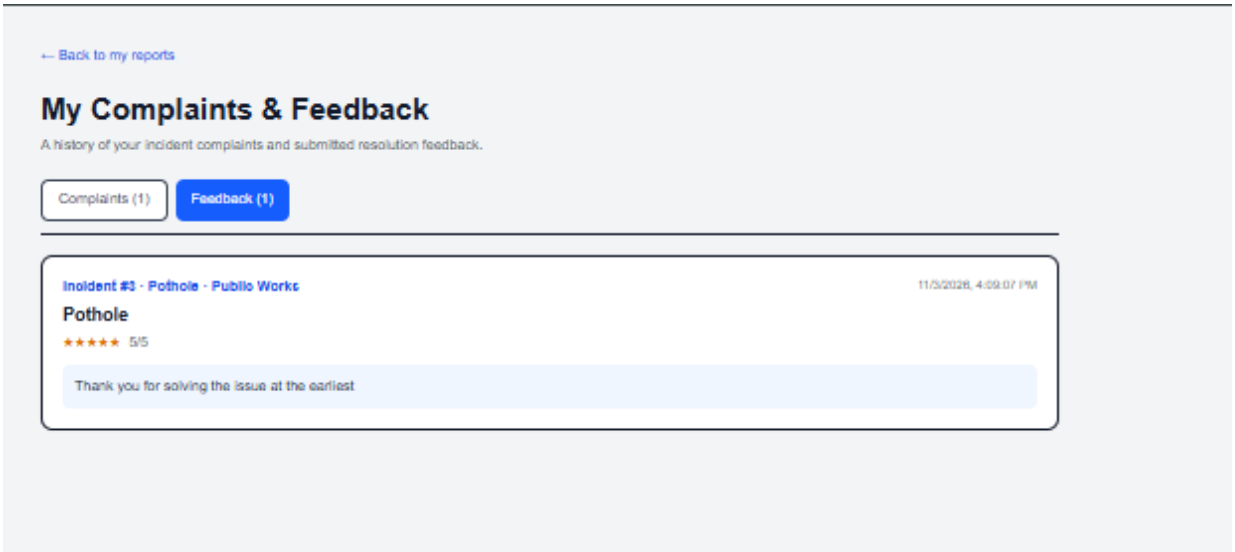


Figure 7. Resident progress page — activity counts and incident status history.





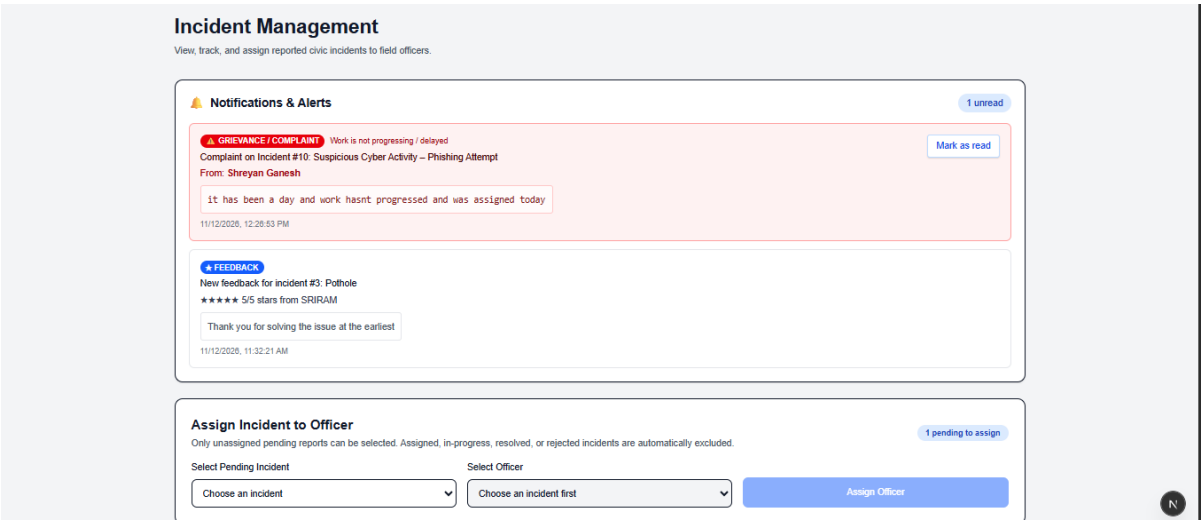


Figure 8. Administrator incident management with feedback and complaint alerts and unread notification count.

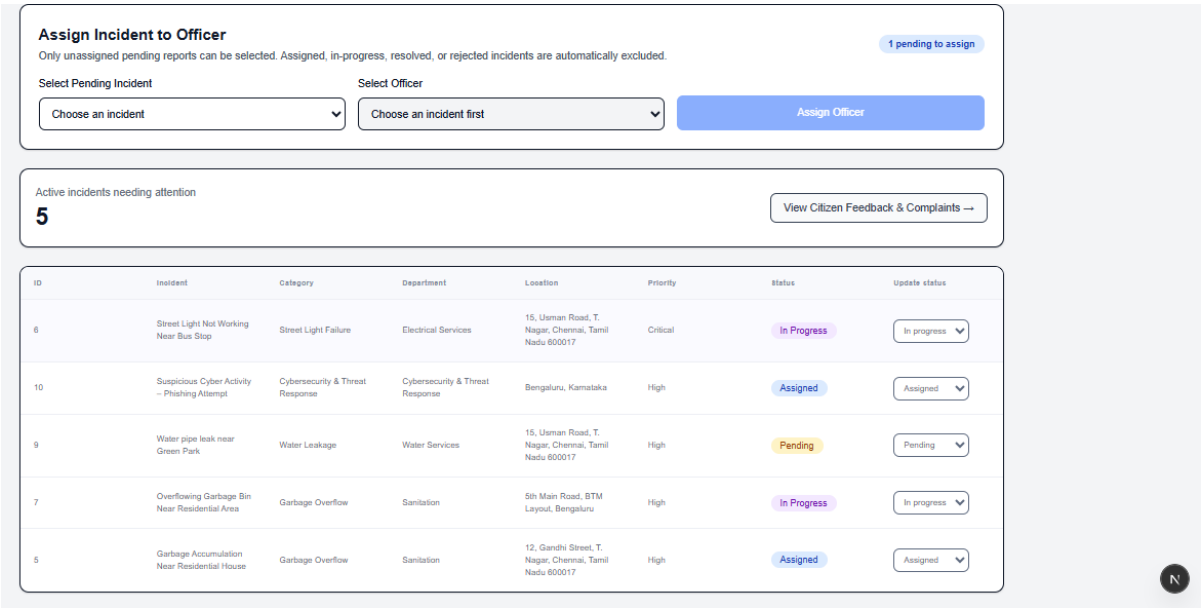


Figure 9. Administrator assignment controls, active-incident count and incident priority/status table.

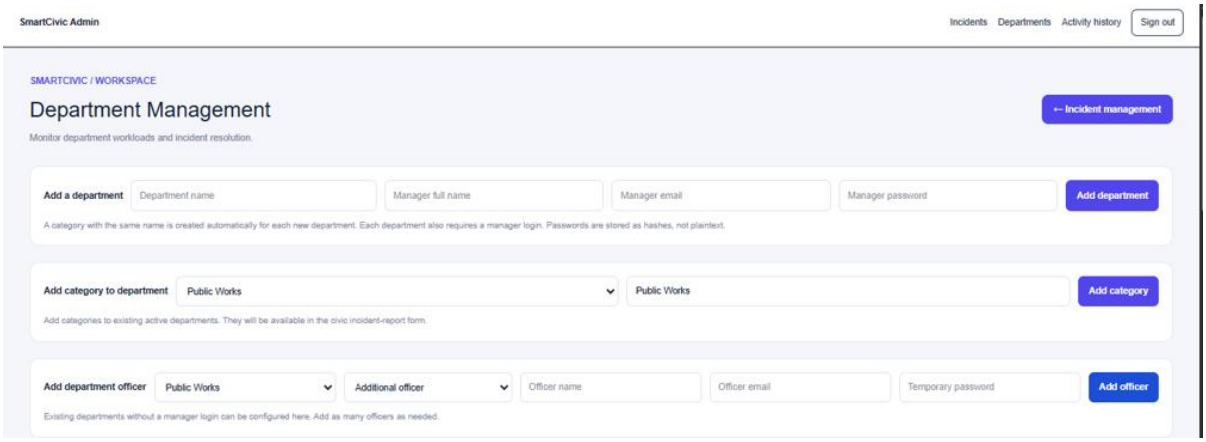


Figure 10. Department management — adding departments, managers, categories and officers.



Figure 11. Department summary — manager, officers and incident totals.

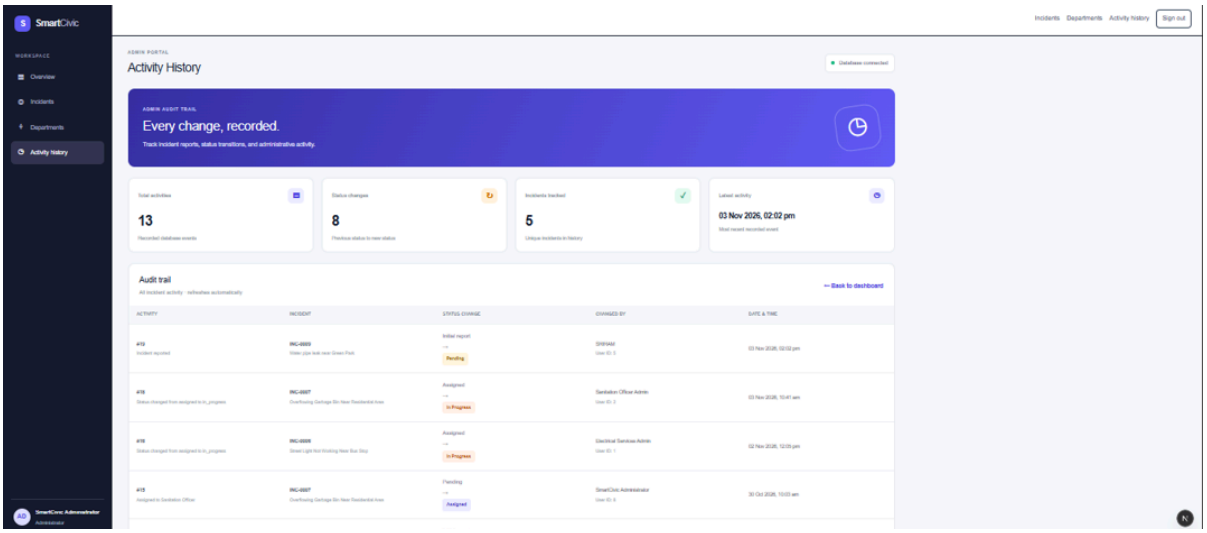


Figure 12. Administrator activity history - audit events and status changes.

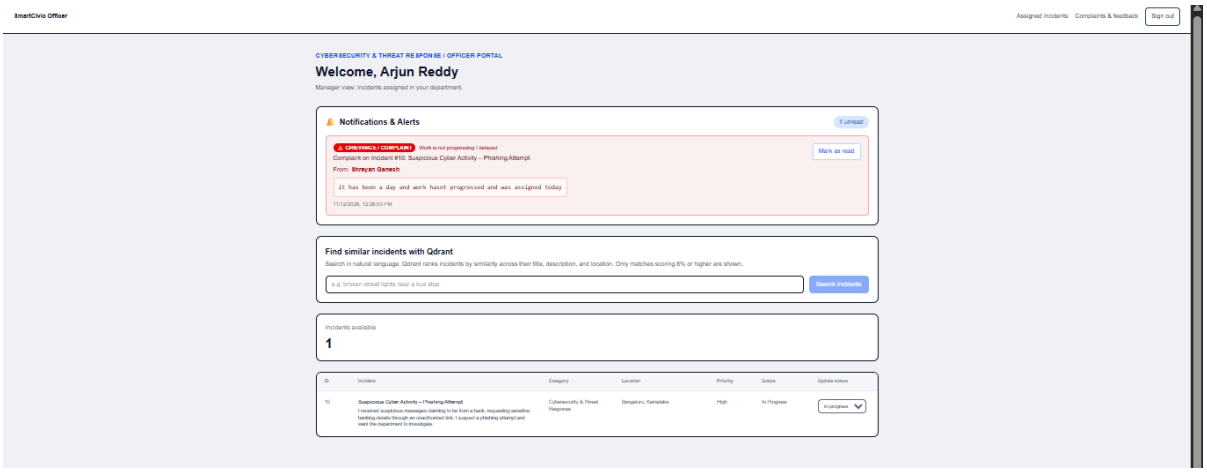


Figure 13. Updated officer portal - notifications, Qdrant similarity search and assigned incidents.

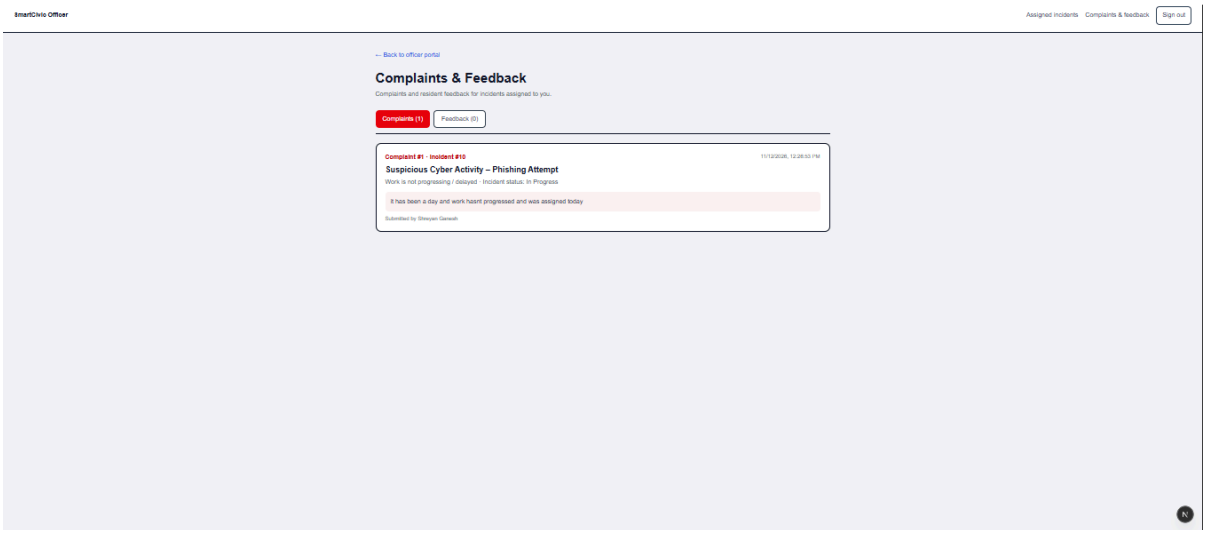
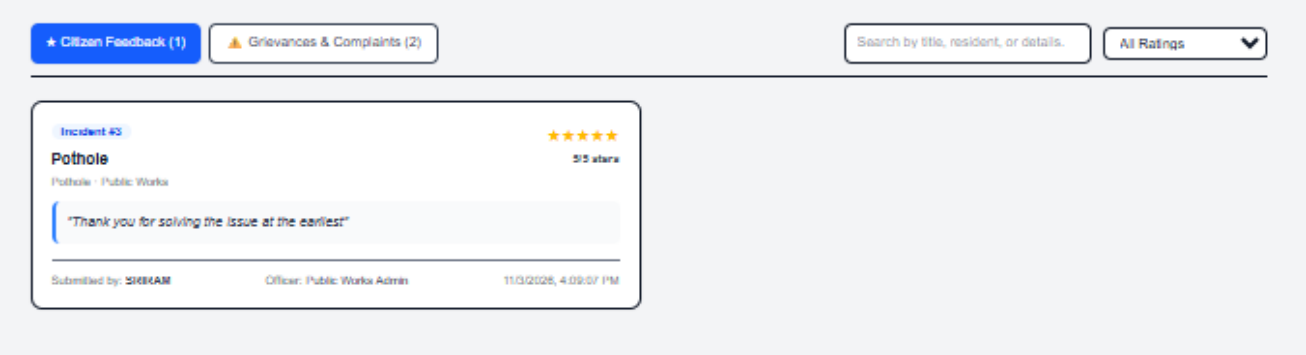


Figure 14. Officer Complaints & Feedback - complaints and resident feedback for incidents assigned to the officer.



Figure 15. Civic Feedback & Escalations - notification panel, average rating, feedback totals and active complaints.



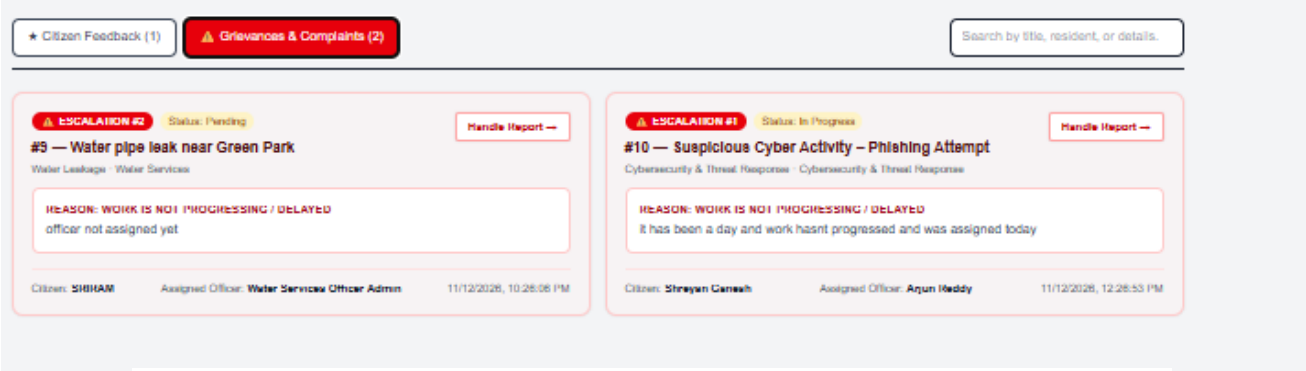


Figure 16. Citizen feedback list - incident, rating, comment, submitter and officer information.

10.1 Application Workflow Demonstration

1. Sign in through the Civic, Administrator or Officer portal.
2. Submit an incident as a resident and confirm that it appears in the resident's incident list.
3. Open administrator incident management, review the report, set priority and assign an officer.
4. Sign in as the assigned officer and update the incident status. Show the corresponding status-history record.
5. For an unresolved incident, submit a complaint as the resident. Confirm the complaint is recorded and the incident status itself remains unchanged.
6. Show the complaint notification in the administrator and assigned officer portals, then mark it as read.
7. Open Civic Feedback & Escalations to show the separate citizen-feedback and grievances/complaints views.

8. After resolving an incident, submit resident feedback and show the resulting feedback notification.

11. Evaluation Rubrics – Completion Status

The table below follows the evaluation categories in the PES University DBMS project template. The status reflects the implementation evidence reviewed for this report. Source inspection and screenshots are not treated as substitutes for a fresh automated test run.

#	Criteria	Evidence in SmartCivic	Status
1	Database Design	Conceptual ERD, relationships and constraints.	Implemented
2	Database Creation	PostgreSQL migrations 001–009, including relational and notification/complaint tables; Qdrant collection configuration.	Implemented in source
3	SQL Operations and Queries	DDL, DML, joins, aggregates, CTE, correlated/NOT EXISTS checks, transactions and parameterized queries.	Implemented in source
4	Advanced Database Concepts	Trigger/function, full-text search, GIN/B-tree indexes, summary view, row locks, vector search and synchronization tracking.	Implemented in source
5	Web Application and Database Integration	Next.js pages and API handlers connected to PostgreSQL and Qdrant.	Implemented in source
6	Authentication and Authorization	Resident, administrator and officer roles; signed sessions; role- and department-scoped access.	Implemented in source
7	Documentation, Git/GitHub, Demonstration, Testing and Originality	Project report, source structure, screenshots and project division details. Commit history remains the evidence for individual contributions.	Documentation updated; testing should be rerun

11.1 Current Feature Verification Checklist

Feature	Evidence / verification point	Status
Resident incident reporting	Incident API and resident portal are present in the supplied source.	Implemented
Priority and officer assignment	Administrator assignment interface and assignment API are present.	Implemented
Officer status updates	Officer portal and status-history workflow are present.	Implemented
Resident feedback	Feedback API and administrator feedback view are present.	Implemented
Complaint escalation	Resident-only submission, ownership check and unresolved-incident rule are present.	Implemented
Notifications	Feedback and complaint notifications; recipient-scoped read updates.	Implemented
Qdrant similarity search	384-dimensional feature-hash vectors and Qdrant search operations are present.	Implemented

Conclusion

SmartCivic provides a complete database-backed workflow for civic incident reporting and follow-up. PostgreSQL stores the authoritative records and enforces relationships, constraints and transactions. The application supports resident reporting, administrative review, department and officer assignment, status tracking, feedback and complaint escalation. Notifications help administrators and assigned officers respond to new feedback and complaints. Qdrant adds similarity ranking while PostgreSQL continues to control access to incident data.

Future Enhancements

- Enforce notification reference integrity with a CHECK constraint so exactly one of feedback_id or complaint_id is populated.
- Define a complete complaint review lifecycle with clear statuses, review responsibility, resolution remarks and timestamps.
- Introduce service-level monitoring and incident-age reports to help administrators identify delayed cases and assess departmental workload.
- Evaluate pretrained multilingual embeddings against the current feature-hash vectorizer using representative civic incident queries and relevance measures.
- Strengthen PostgreSQL?Qdrant synchronization with retry handling, failure monitoring and periodic reconciliation of indexed incidents.
- Expand automated integration and regression tests for role-scoped access, complaint ownership, feedback eligibility, notification isolation and concurrent status updates; document and test backup and recovery procedures.

References and Project Sources

- SmartCivic application source code and database migration files 001–009 supplied for this report.
- SmartCivic project documentation, README and project division note supplied with the source.
- PES University UE23CS351A DBMS Project Template (Aug–Dec 2026).
- Qdrant collection and point-operation structure used by the project implementation.